

# 即時校園地圖通知系統 — 書面報告

---

## 摘要

本專題實作一套即時校園地圖通知系統，讓使用者能在地圖上回報突發事件（如交通事故、施工、人群聚集等），系統自動根據事件座標，只通知半徑範圍內的使用者。系統採用微服務架構，以三個獨立 FastAPI 服務分別處理定位更新、事件管理與即時通知，搭配 Redis GEO 做空間查詢、Redis Pub/Sub 做事件推播，前端使用 React + Leaflet 呈現互動式地圖。部署方面使用 Docker Compose 容器化、nginx 反向代理統一入口、Cloudflare Tunnel 對外提供 HTTPS 連線。壓力測試結果顯示系統在 1000 人同時上線的規模下，事件建立、留言與查詢的成功率達 100%，整體吞吐量 271 RPS。

**關鍵字：**即時通知、地理圍欄、微服務、Redis GEO、WebSocket、Docker

---

## 一、動機與背景

校園內資訊傳遞長期面臨以下問題：

- 資訊分散：**臨時事件（如教室異動、設備故障、道路封閉）通常透過紙本公告、Email 或社群群組傳遞，學生與教職員不容易即時掌握。
- 缺乏地緣關聯：**傳統公告系統不會根據使用者所在位置過濾資訊，導致遠端使用者收到無關通知，而附近的使用者反而錯過重要訊息。
- 反應遲緩：**從事件發生到公告發布，通常需要數小時甚至數天的行政流程。
- 缺乏互動：**公告通常是單向傳播，無法讓現場使用者回報最新狀況或互相交流。

本系統旨在解決上述問題，提供一個基於地理位置的即時事件通知平台。使用者只需開啟網頁，系統便會根據目前所在位置，只推送相關的事件通知，實現「對的人、在對的地方、收到對的資訊」。

---

## 二、需求分析

### 2.1 功能性需求

| 需求編號  | 需求描述                          | 優先順序 |
|-------|-------------------------------|------|
| FR-01 | 使用者可在地圖上即時查看自身位置              | 高    |
| FR-02 | 使用者可透過快選按鈕或自訂輸入發布事件           | 高    |
| FR-03 | 事件發布後，系統自動通知半徑內的使用者           | 高    |
| FR-04 | 事件可設定有效期限，過期自動消失              | 中    |
| FR-05 | 使用者可對事件留言互動                   | 中    |
| FR-06 | 使用者離線時通知暫存，重新上線後補發            | 中    |
| FR-07 | 事件分為一般（info）與緊急（urgent）兩種嚴重程度 | 高    |
| FR-08 | 每個事件可設定通知範圍（100-2000 公尺）      | 中    |

### 2.2 非功能性需求

| 需求編號   | 需求描述            | 目標值                            |
|--------|-----------------|--------------------------------|
| NFR-01 | 系統應支撐 500 人同時上線 | 各端點成功率 $\geq 98\%$             |
| NFR-02 | 緊急事件推播延遲        | 發布後 2 秒內通知                     |
| NFR-03 | API 回應時間        | P95 $< 500\text{ms}$           |
| NFR-04 | 系統記憶體佔用         | $\leq 500\text{MB}$            |
| NFR-05 | 瀏覽器相容性          | 支援主流瀏覽器（Chrome、Firefox、Safari） |

## 三、系統架構

### 3.1 整體架構

系統採用前後端分離的微服務架構，共由以下元件組成：

```
瀏覽器 → nginx (:8080→:8095) → Location Service (:8001, 4 workers) → Redis GEOADD
                                     → Event Service (:8002, 4 workers) → Redis LIST + PUBLISH
                                     ← Notification Service (:8003, 1 worker) ← Redis SUBSCRIBE
瀏覽器 ← nginx (/ws/*) ←
```

資料流： - 位置更新：前端定期上傳 GPS 座標 → Location Service → Redis GEOADD -  
事件推播：前端發布事件 → Event Service → Redis PUBLISH → Notification Service  
(訂閱 channel) → 本地 GEOSEARCH → WebSocket 推播給附近使用者 - 地圖更新：  
前端收到 WebSocket 通知後，在地圖上新增事件標記

### 3.2 微服務設計

系統將功能拆分為三個獨立微服務，各自負責單一領域：

**Location Service** (port 8001, 4 gunicorn workers) - 接收 `POST /locations`  
上傳使用者 GPS 座標 - 將座標存入 Redis GEO 結構 ( `GEOADD` ) - 提供 `GET /locations/nearby`  
查詢指定座標半徑內的使用者 - 為系統中吞吐量最高的服務，因此配置 4 個 worker 處理併發請求

**Event Service** (port 8002, 4 gunicorn workers) - 接收 `POST /events` 建立事件 (含幂等性防重複處理) - 事件持久化至 Redis LIST (保留最新 100 筆) - 透過 Redis Pub/Sub 發布事件通知 (取代原本的 HTTP fanout) - 支援事件過期 ( `expires_in` 欄位, 1-1440 分鐘, 預設 30) - 提供留言功能 ( `POST/GET /events/{id}/comments` )

**Notification Service** (port 8003, 1 gunicorn worker) - 管理 WebSocket 連線 ( `WS /ws/{user_id}` ) - 訂閱 Redis Pub/Sub channel, 收到事件後執行本地 GEOSEARCH 篩選附近使用者 - 透過 WebSocket 即時推播通知 - 離線通知佇列: 使用者離線時存入 Redis LIST `pending:{user_id}`, 重連後回放 - App-level ping/pong 心跳 (每 15 秒), 偵測殭屍連線

### 3.3 為何選擇 Redis Pub/Sub 取代 HTTP Fanout

在架構演進過程中，事件推播經歷了重大改動。原先 Event Service 收到事件後，會查詢附近使用者，再逐一發送 HTTP POST 給 Notification Service (即 HTTP fanout)。在 500 人壓測時，這種方式導致事件建立成功率僅 8.1%。

改用 Redis Pub/Sub 後：- Event Service 只需執行一條 `PUBLISH` 命令 (微秒級)，不再逐一 HTTP 呼叫 - Notification Service 訂閱 channel 後，在本地執行 GEOSEARCH 篩選附近使用者 - 事件建立成功率從 8.1% 提升至 **100%** (500 人壓測)

### 3.4 共用模組

三個微服務共用以下模組 ( `backend/shared/` )：

- `schemas.py` — Pydantic 資料模型：`LocationUpdate`、`EventCreate` (含 `expires_in`)、`EventNotification`、`EventRecord`、`Comment`
- `config.py` — 環境變數設定 (REDIS\_URL、CORS、冪等 TTL、事件歷史上限)
- `redis_client.py` — 共用 Redis 連線池 (`max_connections=20`, `socket_timeout=5s`)
- `cors.py` — CORS middleware 統一設定

## 四、技術選型

### 4.1 前端技術

| 技術                      | 用途    | 選擇原因                   |
|-------------------------|-------|------------------------|
| React 18                | UI 框架 | 組件化開發、豐富生態系、易於維護       |
| Vite                    | 建構工具  | 開發伺服器快速 (HMR)、打包效率高    |
| TypeScript              | 型別系統  | 編譯期型別檢查，降低執行期錯誤        |
| Leaflet                 | 地圖元件  | 輕量 (~40KB)、開源、支援多種圖磚來源 |
| browser Geolocation API | 定位功能  | 無需額外 SDK，瀏覽器原生支援       |

## 4.2 後端技術

| 技術                         | 用途          | 選擇原因                        |
|----------------------------|-------------|-----------------------------|
| Python FastAPI             | Web 框架      | 原生 async、自動 API 文件、高效能      |
| Gunicorn + Uvicorn workers | ASGI 伺服器    | 多 worker 利用多核 CPU、生產環境穩定    |
| Redis 7                    | 資料儲存        | GEO 空間查詢、Pub/Sub 推播、LIST 佇列 |
| WebSocket                  | 即時通訊        | 伺服器主動推送、低延遲雙向通訊             |
| httpx                      | HTTP client | 非同步、連線池、取代同步 requests       |

## 4.3 基礎設施

| 技術                | 用途   | 選擇原因                             |
|-------------------|------|----------------------------------|
| Docker Compose    | 容器編排 | 單機部署簡單、服務隔離、可重現環境                |
| nginx             | 反向代理 | 統一入口、靜態檔託管、WebSocket upgrade     |
| Cloudflare Tunnel | 對外連線 | 免開 inbound port、自動 HTTPS、DDoS 防護 |

## 4.4 為何選擇 Redis GEO 而非 PostGIS

本系統的核心查詢是「找出座標半徑內的使用者」，Redis GEO 提供 `GEOADD` 和 `GEOSEARCH` 命令，可在微秒級完成此操作。相較之下：

- PostGIS 功能更強大（支援複雜空間查詢），但對本專題的需求而言過重
- Redis 是記憶體操作，延遲遠低於磁碟資料庫
- 本系統的位置資料屬於短期暫存，不需要持久化，Redis 的 TTL 機制正好符合

## 五、系統實作

### 5.1 位置更新流程

1. 前端透過瀏覽器 Geolocation API 取得 GPS 座標

2. 每隔 1.5 秒向 Location Service 發送 `POST /locations`
3. Location Service 使用 Redis `GEOADD` 寫入座標
4. 同時設定 `last_seen` TTL (60 秒) , 超時視為離線

## 5.2 事件推播流程

1. 使用者在前端選擇事件類型 (快選按鈕或自訂輸入) , 填寫描述、嚴重程度、有效期限
2. 前端呼叫 `POST /events` , 附帶 `client_event_id` ( `crypto.randomUUID()` ) 確保冪等性
3. Event Service :
4. 使用 Redis `SET NX` 做 5 分鐘去重 (防止重複推播)
5. 計算 `expires_at = now + expires_in minutes`
6. 將事件存入 Redis LIST (保留最新 100 筆)
7. 發布 `PUBLISH event_channel {event_data}` (微秒級)
8. Notification Service :
9. 訂閱 `event_channel` , 收到訊息後執行 `GEOSEARCH` 查詢附近使用者
10. 對每個在線使用者透過 WebSocket 推送通知
11. 對離線使用者, 將通知存入 `pending:{user_id}` 佇列

## 5.3 留言系統

- 每個事件支援留言功能, 資料存於 Redis LIST (每事件最多 100 則)
- `POST /events/{id}/comments` : 新增留言
- `GET /events/{id}/comments` : 查詢留言列表
- 前端提供 `EventDetailPanel` 元件, 顯示事件詳情與留言串

## 5.4 前端 UI 設計

前端包含以下核心元件 :

- **MapView** : Leaflet 地圖, 顯示使用者位置、事件標記
- **EventForm** : 事件發布表單, 含 4×2 常用事件快選格 (交通事故、施工中、人群聚集、設備故障、道路封閉、其他) + 有效期限下拉選單 (15 分鐘、30 分鐘、1 小時、2 小時、4 小時、8 小時、24 小時)
- **NotificationBanner** : 通知橫幅, 區分緊急 (紅色) 與一般 (藍色) 事件, 顯示距離
- **EventDetailPanel** : 事件詳情面板 + 留言輸入框

## 5.5 安全性設計

- **冪等性**：Event Service 使用 `client_event_id` + Redis `SET NX` + TTL 防重複處理
- **Docker 非 root**：所有容器以 `appuser` 非 root 帳號執行
- **CORS** 設定：僅允許指定來源 ( `map2.avision-gb10.org` )
- **WebSocket** 自動協定偵測：根據頁面協定自動選擇 `ws:` 或 `wss:`

# 六、壓力測試與效能分析

## 6.1 測試方法

使用 Python `asyncio` + `httpx` 撰寫壓力測試腳本 ( `stress_test.py` )，模擬多用戶並發場景：

- **Location**：每 1.5 秒上傳 GPS 座標
- **Event**：每 3-8 秒隨機建立事件
- **Query**：每 2-5 秒查詢事件列表
- **Comment**：每 5-15 秒新增留言

測試在 DGX Spark (20 核 ARM CPU、119GB RAM) 上執行，服務以 Docker Compose 部署。

## 6.2 壓力測試結果

| 規模     | Location | Event       | Comment     | Query       | 總 RPS |
|--------|----------|-------------|-------------|-------------|-------|
| 200 人  | 99.1%    | 99.0%       | 100%        | 100%        | 112   |
| 500 人  | 98.4%    | <b>100%</b> | 99.1%       | <b>100%</b> | 277   |
| 1000 人 | 95.5%    | <b>100%</b> | <b>100%</b> | <b>100%</b> | 271   |

## 6.3 瓶頸分析與解決過程

問題一：事件推播 **Fanout** 瓶頸

症狀：500 人壓測時，Event Service 成功率僅 8.1%。

根因：Event Service 收到事件後，需對每個附近使用者發送 HTTP POST 給 Notification Service (HTTP fanout)。當附近使用者數量多時，同步 HTTP 呼叫造成嚴重延遲。

嘗試的方案：1. FastAPI BackgroundTasks — 在 gunicorn pre-fork worker 中，response 發送後 event loop 被回收，背景任務無法執行 2. asyncio.ensure\_future — 同樣因 gunicorn worker 生命週期問題失敗 3. 單 worker uvicorn — 效能更差

最終方案：Redis Pub/Sub 解耦 - Event Service 只執行 `PUBLISH event_channel {data}` (一條 Redis 命令，微秒級) - Notification Service 訂閱 channel，在本地執行 GEOSEARCH 篩選 + WebSocket 推播 - 消除了跨服務 HTTP 呼叫，Event Service 回應時間大幅下降

效果：Event 成功率從 8.1% 提升至 **100%**，整體吞吐量從 112 RPS 提升至 277 RPS。

問題二：**Location Service** 效能瓶頸

症狀：1000 人壓測時 Location Service 成功率降至 95.5%。

根因：Location Service 是吞吐量最高的服務 (每 1.5 秒接收所有使用者的座標更新)，即使 4 個 gunicorn workers 也接近極限。

可能的改善方向：- 增加 worker 數量 (目前 4，可提升至 8-16) - 使用 Redis pipeline 批次寫入 - 導入訊息佇列 (如 Redis Stream) 緩衝寫入壓力

---

## 七、問題與解決

### 7.1 Gunicorn + Async 背景任務不相容

問題：嘗試用 FastAPI BackgroundTasks 做非同步 fanout，但 gunicorn pre-fork 模式下，HTTP response 發送後 worker 的 event loop 會被回收，導致背景任務靜默失敗。



**解決：**改用 Redis Pub/Sub，將 fanout 邏輯從 Event Service 完全移除，改由 Notification Service 訂閱處理。這不僅解決了背景任務問題，也根本性地消除了跨服務 HTTP 呼叫的效能瓶頸。

## 7.2 WebSocket 連線管理

**問題：**WebSocket 長連線可能因網路不穩定而中斷，產生「殭屍連線」佔用資源。

**解決：** - App-level ping/pong 心跳（每 15 秒），超時未回應則關閉連線 - 前端自動偵測 ws: / wss: 協定，支援 HTTPS 環境 - 離線通知佇列：使用者斷線期間的通知存入 Redis LIST，重連後自動回放

## 7.3 Docker npm ci 失敗

**問題：**Dockerfile 中 `npm ci` 因 lock file 版本不匹配而失敗。

**解決：**改用 `npm install`，雖然犧牲了嚴格版本鎖定，但確保建構流程穩定。

## 7.4 Port 衝突

**問題：**原本規劃使用 port 8090 對外提供服務，但該 port 已被其他服務佔用。

**解決：**改用 port 8095，同步更新 docker-compose.yml 和 Cloudflare Tunnel 設定。

---

# 八、分工

本專題由四位成員協作完成，分工如下：

- **成員 A：**Web App 前端開發 — React 組件設計、Leaflet 地圖整合、瀏覽器 Geolocation API、UI/UX 設計
  - **成員 B：**後端 API 開發 — Event Service 事件管理、商業邏輯、Pydantic Schema 設計
  - **成員 C：**Redis 整合與即時推播 — Redis GEO 空間查詢、WebSocket 連線管理、Pub/Sub 推播機制
  - **成員 D：**容器化、部署與壓測 — Docker Compose 設定、nginx 反向代理、Cloudflare Tunnel、壓力測試腳本撰寫與分析
-

## 九、結論

本專題成功實作了一套即時校園地圖通知系統，具備以下特點：

1. **基於地理位置的精準推播**：利用 Redis GEO 空間查詢，只通知事件半徑內的使用者，避免無關通知造成的資訊噪音。
2. **微服務架構具擴展性**：三個獨立服務可根據各自的負載特性獨立擴展（Location Service 需要 4 workers 應付高頻座標更新，Event Service 同樣配置 4 workers，Notification Service 因 WebSocket sticky connection 特性保持單 worker）。
3. **Redis Pub/Sub 消除推播瓶頸**：從 HTTP fanout 演進到 Redis Pub/Sub，事件建立成功率從 8.1% 提升至 100%。
4. **壓力測試驗證穩定性**：在 1000 人同時上線的規模下，Event、Comment、Query 三個端點成功率均達 100%，系統整體吞吐量 271 RPS。
5. **完整的容器化部署**：Docker Compose + nginx + Cloudflare Tunnel，一條指令即可完成部署，對外提供 HTTPS 安全連線。

未來可進一步改善的方向包括：增加使用者認證系統、支援多校園擴展、開發行動 App 版本、以及導入 AI 輔助事件分類。

---

## 十、參考資料

1. FastAPI 官方文件 — <https://fastapi.tiangolo.com/>
  2. Redis GEO 命令 — <https://redis.io/commands/geoaddd>
  3. Redis Pub/Sub — <https://redis.io/docs/interact/pubsub/>
  4. Leaflet 地圖框架 — <https://leafletjs.com/>
  5. React 官方文件 — <https://react.dev/>
  6. Docker Compose 文件 — <https://docs.docker.com/compose/>
  7. WebSocket API — <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
  8. Cloudflare Tunnel — <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
  9. Gunicorn 伺服器 — <https://gunicorn.org/>
  10. Geolocation API — [https://developer.mozilla.org/en-US/docs/Web/API/Geolocation\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API)
-

本報告為即時校園地圖通知系統專題之書面文件。線上展示：<https://map2.avision-gb10.org>